

PhishGuard Final Technical Demo Script & Project Breakdown

This document provides a highly detailed, technical explanation of the PhishGuard system, tailored for a live-action demonstration. It assumes the browser extension is actively scanning real websites, and it explains the underlying machine learning algorithms, embedding correlations, and data flows.

The speaking roles are divided equally among Mallikarjun, Navjyoth, Viharika, and Pranav.

Part 1: Deep Technical Project Explanation

1. System Architecture

PhishGuard operates on a decoupled client-server architecture: - **Client (Browser Extension)**: Developed in JavaScript (`background.js`, `manifest.json`), this extension acts as a silent agent. When a user visits a website, it traverses the DOM (Document Object Model) to dynamically extract the URL, scrape visible text, download images, and capture any audio/video elements present. - **Server (Flask Backend)**: The extracted payload is serialized as JSON and posted to our secure Python Flask backend API (`server.py`).

2. The 5 Machine Learning Modalities

The backend routes the payload to five specialized ML pipelines concurrently:

1. **URL (`phishurl.py`)**: An ensemble model utilizing a `VotingClassifier` (Random Forest, Gradient Boosting, and Logistic Regression). It extracts over 50 structural features (entropy, alphanumeric ratio, domain depth) and cross-references them. We specifically trained it to recognize over 500+ legitimate Indian government and banking websites to prevent false positives.
2. **Text (`phishtext.py`)**: Uses advanced Natural Language Processing (NLP) models generating 768-dimensional embeddings to detect semantic urgency, deceptive language, and social engineering patterns common in phishing emails or landing pages.
3. **Image (`phishimage.py`)**: Utilizes Computer Vision (CNNs) to detect brand spoofing. It extracts visual features into 64-dimensional embeddings, looking for slight alterations in logos or text hidden within images (OCR) designed to bypass text-based spam filters.
4. **Audio (`phishaudio.py`)**: Transcribes audio into text and analyzes voice clips for digital artifacts that indicate AI-generated deepfakes or vishing (voice phishing) attempts.
5. **Video (`phishvideo.py`)**: Extracts frames sequentially to check for visual manipulation and deepfake face-swapping, while running the audio track through the audio pipeline.

3. The Core Fusion Engine (`phishfusion.py`)

This is the heart of the system. Traditional systems vote simply, but our Fusion Engine uses **Embedding Correlation (Cosine Similarity)** and **Dynamic Weighting**. - **Dynamic Weighting:** Base weights are assigned (URL: 1.5, Text: 1.2, Video: 1.1, Image: 1.0, Audio: 1.0). These are dynamically multiplied by the *confidence* score of each model's prediction. - **Cosine Similarity:** The engine aligns the 768-dim NLP vectors and 64-dim Vision vectors, calculating cosine similarity. If the models “agree” (e.g., the text sounds urgent AND the URL looks malicious), it generates an **Agreement Multiplier** that boosts the final risk score. If they disagree, it lowers the score.

Part 2: Technical Demo Presentation Script

Instructions: Speak confidently about the algorithms. You are demonstrating a live, active system.

Speaker 1: Mallikarjun

Role: System Architecture, The Fusion Engine, & Introduction - **Mallikarjun:** “Good morning respected panel. We present **PhishGuard: A Multi-Modal Phishing Detection System**. I am Mallikarjun, and my team includes Navjyoth, Viharika, and Pranav.” - **Mallikarjun:** “Modern phishing has evolved beyond bad URLs; attackers now use spoofed images, hidden text, and even deepfake audio. Traditional single-vector systems fail here. To solve this, we built a real-time browser extension that feeds a cloud-hosted Flask backend. Our backend runs 5 distinct ML pipelines concurrently.” - **Mallikarjun:** “But the true innovation is our **Core Fusion Engine**. It doesn't just average the scores. It performs high-dimensional embedding correlation using Cosine Similarity. If our text model and image model embeddings show a high mathematical correlation—meaning they agree it's an attack—an Agreement Multiplier dynamically boosts our final threat confidence. I'll now pass it to Navjyoth to detail our first line of defense.”

Speaker 2: Navjyoth

Role: URL Ensemble Model & NLP Text Analysis - **Navjyoth:** “Thank you, Mallikarjun. Our first modality is URL analysis. Unlike basic blocklists, our `phishurl` module extracts over 50 structural features—like entropy and path depth—and feeds them into an ensemble `VotingClassifier` utilizing Random Forest and Gradient Boosting. We specifically trained it to recognize over 500 legitimate Indian institutional websites to eliminate false positives.” - **Navjyoth:** “Simultaneously, our `phishtext` module runs on the page's DOM content. We use deep Natural Language Processing generating 768-dimensional embeddings to perform semantic analysis. It doesn't just look for keywords; it understands

the *context* of urgency, financial threats, or social engineering. Now, Viharika will explain our visual and audio defenses.”

Speaker 3: Viharika

Role: Computer Vision, Image OCR, & Audio/Video Deepfake Detection - **Viharika:** “Thank you, Navjyoth. Attackers often bypass text filters by embedding text inside images or spoofing logos. Our **phishimage** module uses Convolutional Neural Networks to generate 64-dimensional embeddings of page images. We use computer vision to detect brand inconsistencies and Optical Character Recognition (OCR) to extract hidden text.” - **Viharika:** “Furthermore, to combat next-generation attacks, we implemented audio and video pipelines. Our system extracts media elements from the DOM in real-time, transcribing audio to detect vishing scripts, and running frame-by-frame analysis to identify deepfake manipulations. All five of these model outputs are then routed back to Mallikarjun’s Fusion Engine. Pranav will now demonstrate the live system.”

Speaker 4: Pranav

Role: Live Browser Extension Demo, User Interface, & Conclusion - **Pranav:** “Thank you, Viharika. I will now demonstrate PhishGuard actively scanning websites in real-time.” (*Pranav opens the browser and navigates to a legitimate site*). - **Pranav:** “Here we are on a legitimate banking portal. As the page loads, our extension has already scraped the DOM, extracted the features, and pinged our backend API. The system returns a ‘Safe’ rating, as our URL ensemble and text models found no anomalies.” - **Pranav:** “Now, I will navigate to a live, complex phishing threat.” (*Pranav navigates to the target site and opens the extension UI*). “As you can see in our interactive UI, the system immediately intervened. The UI breaks down the threat matrix: the NLP model flagged semantic urgency, the CV model detected a spoofed logo, and the URL model caught a high-entropy domain path. The Fusion Engine calculated a critical risk score based on the high embedding correlation between these modalities, successfully blocking the user.” - **Pranav:** “In conclusion, PhishGuard provides a highly scalable, mathematically rigorous defense against multi-modal threats. Thank you, the panel is now open for questions.”

Part 3: “Who Did What?” (Panel Q&A Breakdown)

Ensure you stick to these specialized roles if the panel asks about individual contributions:

Mallikarjun (Lead Architect & Core Systems): - *Response:* “I architected the overall system and backend infrastructure. I built the Flask API (**server.py**), integrated the concurrent ML pipelines, and developed the mathematical models for the **Core Fusion Engine** (**phishfusion.py**), specifically

implementing the dynamic weighting and cosine similarity algorithms for the embeddings.”

Navjyoth (NLP & Data Engineering): - *Response:* “I focused on textual data and the URL algorithms. I engineered the feature extraction pipeline for the URL ensemble model (`phishurl.py`) ensuring it recognized Indian institutional domains, and I trained the NLP models for the text module (`phishtext.py`) to detect semantic social engineering patterns.”

Viharika (Computer Vision & Media Analytics): - *Response:* “I led the media analytics. I developed the computer vision models in the image module (`phishimage.py`) for logo spoofing detection, and I designed the complex pipelines required to extract and analyze frames and waveforms for the video and audio modules (`phishaudio.py`, `phishvideo.py`).”

Pranav (Frontend Engineer & API Integration): - *Response:* “I led the frontend architecture and user experience. I built the Chrome extension from scratch, developed the DOM-scraping algorithms (`background.js`) to extract the multi-modal payload, built the interactive ‘Simulation Lab’ UI, and ensured seamless asynchronous communication with our backend API.”